

**YENEPOYA INSTITUTE OF ARTS, SCIENCE,
COMMERCE AND MANAGEMENT A CONSTITUENT
UNIT OF YENEPOYA (DEEMED TO BE UNIVERSITY)**

BALMATTA, MANGALORE

Project – High Level Design

on

CLOUD COST INTELLIGENCE PLATFORM FOR RETAIL

Student Name: MOHAMED RIFAN P

**Course : BCA Cloud Computing And Devops
IBM&TCS**

Reg No : 23BCAICD052

Project Code : PRJN26-118

Industry Mentor: Sumit Kumar Shukla

Guided by: MS NIHARIKA

Table of Contents

1. Introduction	3
1.1. Scope of the document	3
1.2. Intended Audience	3
1.3. System overview	3
2. System Design	4
2.1. Application Design	4
2.2. Process Flow	4
2.3. Information Flow	4
2.4. Components Design	5
2.5. Key Design Considerations	5
2.6. API Catalogue	5
3. Data Design	6
3.1. Data Model	6
3.2. Data Access Mechanism	6
3.3. Data Retention Policies	6
3.4. Data Migration	7
4. Interfaces	7
5. State and Session Management	8
6. Caching	8
7. Non-Functional Requirements	9
7.1. Security Aspects	9
7.2. Performance Aspects	9
8. References	10

1. Introduction

Retail Cloud Cost Intelligence (RCCI) is an AI-powered Streamlit platform providing visibility into multi-cloud spending for retail departments (Grocery, Fashion, Electronics, Supply Chain) across services (Compute, Storage, Analytics, AI/ML). It transforms billing CSV files into dashboards, insights, and optimization recommendations using inmemory Pandas/NumPy processing.

1.1 Scope

This document covers system architecture, data flow, component design, API structure, security, and performance. Low-level coding details excluded.

1.2 Intended Audience

Technical Architects, IBM Mentors, Faculty, DevOps Engineers

1.3 System Overview

- Executive Dashboard | Intelligent Agents | Insights Engine | Optimization Analyzer | Data Explorer

2. System Design

2.1 Application Design (Three-Layer)

Layer	Components
Presentation	Streamlit UI (Overview, Agents, Insights, Optimization, Data)
Business Logic	Data Pipeline, Agent Manager, Forecast Engine
Data Processing	Pandas DataFrame (in-memory, session-scoped)

2.2 Process Flow

text

CSV Upload → Validation → Cleaning → Filters → Agents → Dashboard

2.3 Information Flow

text

CSV File → Pandas DataFrame → Filtered DataFrame → Agent Results → UI Dashboard

Component	Function
Data Ingestion	CSV upload, validation, normalization
Filter Engine	Date, department, service, region filters

Agent Orchestrator	Manages 5 analysis agents
Component	Function
Synthetic Generator	Creates test data

2.4 Components Design

2.5 Key Design Considerations

Decision	Rationale
Stateless Processing	No DB overhead, privacy, simple scaling
Modular Agents	Independent development, failure isolation
INR Currency Support	Lakh/Crore formatting for Indian users

2.6 API Catalogue

Function	Purpose	Error Handling
<code>coerce_schema()</code>	Normalize columns	Raises SchemaError
<code>load_data()</code>	Parse CSV	Returns None on failure
<code>rolling_zscore()</code>	Anomaly detection	Zeros if insufficient data
<code>month_end_projection()</code>	Forecasting	Falls back to linear regression
	Optimization	Message if <7 days data

idle_candidates()		
-------------------	--	--

3. Data Design

3.1 Data Model

Column	Type	Required
Date	Date	Yes
Retail_Department	String	Yes
Service_Type	String	Yes
Region	String	Yes
Cost_INR	Float	Yes
Usage_Units	Float	No

3.2 Data Access Mechanism

CSV → Pandas read_csv() → @st.cache_data → Filtered DataFrame

3.3 Data Retention Policies

- Storage: RAM only | Session duration (60 min timeout) | No backup

3.4 Data Migration

Scenario	Approach
CSV format changes	Auto-rename columns via schema coercion
Future DB migration	Export CSV → Import to database
Rollback	Revert to CSV-only mode

4. Interfaces

Tab	Content
Overview	KPI cards, trend charts, breakdowns
Agents	Findings from 5 analysis agents
Insights	Anomalies, forecasts, variance
Optimization	Idle resources, savings calculator
Data	Raw view + export (CSV, JSON, PNG)

5. State and Session Management

```
python st.session_state = {
  "raw_df": DataFrame,
  "filtered_df": DataFrame,
  "filters": {...},
  "agent_results": {...}
}
```

- Isolated sessions | 60-minute timeout | No cross-user sharing

6. Caching

Function	Cache	Hit Rate
load_data()	@st.cache_data	70-85%
resample_cost()	@st.cache_data	

7. Non-Functional Requirements

7.1 Security Aspects

Threat	Mitigation
Malicious CSV	Sanitize strings
Large file DoS	50MB limit
XSS	Streamlit auto-escaping
Session hijacking	HTTPS + secure cookies

7.2 Performance Aspects

Operation	Target
CSV parse (50K rows)	<2 sec
Dashboard load	<3 sec
All 5 agents	<2 sec
Peak memory	<500 MB
Max rows	100,000
Concurrent users	50

8. References

- Streamlit, Pandas, NumPy Documentation
- IBM Cloud Cost Management
- OWASP Guide | FinOps Foundation